RAJALAKSHMI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240070120
NAME	JAGANNATHAN S
PROJECT TITLE	Disaster Damage Assessment using ML
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Mrs K. FOUZIA SULTHANA (M.TECH M.B.A)

Signature of Faculty In-charge **Signature of HoD**

Internal Examiner **External Examiner**

INTRODUCTION

- Natural disasters such as earthquakes, floods, cyclones, and hurricanes have become increasingly frequent in recent years, causing severe destruction to infrastructure, environment, and human life. Rapid and accurate assessment of damage is essential for effective disaster management and rescue operations.
- Traditionally, damage assessment is carried out manually by experts through field surveys and visual inspections. However, this process is time-consuming, costly, and often delayed, especially in large-scale disasters where immediate response is required.
- With the advancement of Machine Learning (ML) and Computer Vision, it is now possible to automate the damage assessment process using image data. These technologies enable systems to analyze images captured from satellites, drones, or ground-level cameras and classify the severity of damage.
- In this project, a Disaster Damage Assessment System is developed using deep learning techniques. The system uses Convolutional Neural Networks (CNN) to classify images into different categories such as No Damage, Minor Damage, Major Damage, and Destroyed.
- The primary objective of this project is to design an intelligent system that can automatically detect and classify disaster damage, helping authorities take faster and more informed decisions during emergency situations.

PROBLEM STATEMENT

- Disaster management authorities face significant challenges in quickly assessing damage across affected regions. Due to the large geographical areas involved and limited resources, manual inspection methods are inefficient and slow.
- In many cases, delayed damage assessment can lead to ineffective rescue operations, improper allocation of resources, and increased loss of life and property.
- Traditional methods are not scalable and cannot handle real-time analysis. Therefore, there is a need for an automated system that can process large volumes of image data and provide accurate damage classification instantly.
- This project addresses the problem by developing a machine learning-based system that analyzes disaster images and classifies damage levels efficiently, enabling faster and more reliable decision-making.

GOAL

The main goal of this project is to develop a machine learning model capable of analyzing images and classifying disaster damage levels.

The system is designed to categorize images into four classes:

- No Damage
- Minor Damage
- Major Damage
- Destroyed

The project also focuses on preprocessing image data, extracting features, and training a deep learning model using CNN architectures.

Additionally, the system aims to provide a user-friendly interface where users can upload images and receive predictions instantly.

THEORETICAL BACKGROUND

Supervised Learning and Classification

This project is based on supervised learning, where the model is trained using labeled image data. Each image is associated with a specific damage category, making it a multi-class classification problem.

Image Preprocessing

Images are resized to a standard dimension (224×224 pixels) and normalized to improve model performance. Data augmentation techniques such as rotation, flipping, and zooming are applied to increase dataset diversity.

Convolutional Neural Networks (CNN)

CNN is a deep learning model specifically designed for image processing tasks. It extracts important features such as edges, textures, and patterns from images and uses them for classification.

Transfer Learning

Pre-trained models like MobileNetV2 are used to improve performance and reduce training time. These models have already learned useful features from large datasets.

Data Visualization

Graphs and charts are used to analyze dataset distribution and model performance. Accuracy and loss graphs help in evaluating the effectiveness of the model.

ALGORITHM EXPLANATION WITH EXAMPLE

The Convolutional Neural Network (CNN) is a deep learning algorithm widely used for image classification and computer vision tasks. In this project, CNN is used to analyze disaster images and classify them into different damage categories such as No Damage, Minor Damage, Major Damage, and Destroyed.

CNN is highly effective because it can automatically learn important features from images, such as edges, textures, shapes, and structural patterns, without the need for manual feature extraction.

Working Steps in this Project

1. Input image is provided
2. Image is resized and normalized
3. CNN extracts features using convolution layers
4. Features are passed through fully connected layers
5. Model predicts the damage category

Example

Input Image

A photograph of a building taken after an earthquake showing visible structural damage.

Model Processing

- The image is resized and normalized
- CNN detects key features such as:
  → Cracks in walls
  → Broken roofs
  → Collapsed structures
  → Debris around buildings
- Feature maps highlight damaged areas
- Fully connected layers analyze severity

Probability Calculation

- No Damage → Low Probability
- Minor Damage → Moderate Probability
- Major Damage → High Probability
- Destroyed → Very High Probability

Final Prediction

→ **Destroyed (Highest Probability)**

IMPLEMENTATION CODE AND DATASET USED

Dataset Used

Kaggle – https://www.kaggle.com/datasets/jaga12345/disaster-damage-dataset

Category	Description
Dataset Name	disaster_damage_dataset
Domain	Computer Vision / Deep Learning
Objective	Image Classification (Disaster Damage Assessment)
Data Source	Kaggle (Public Dataset)
Data Type	Image Files (.jpg / .png)
Number of Samples	2000+ images (approx.)
Input Type	RGB Images
Image Size	Resized to 224 × 224 pixels
Color Format	3 Channels (Red, Green, Blue)
Classes	4 Classes (Damage Levels)
Class Labels	No Damage, Minor Damage, Major Damage, Destroyed
Output Type	Multi-Class Classification
Dataset Split	Training Set (80%), Testing Set (20%)
Training Data	Used for model learning
Testing Data	Used for model evaluation
Feature Type	Visual Features (edges, textures, patterns)
Label Type	Categorical Labels
Preprocessing	Image resizing, normalization, augmentation
Augmentation Techniques	Rotation, flipping, zooming
Annotation Type	Image-level classification
Target Variable	Damage Category (4 classes)
Data Balance	Approximately balanced across classes
License	Open Source / Educational Use
Version	v1.0

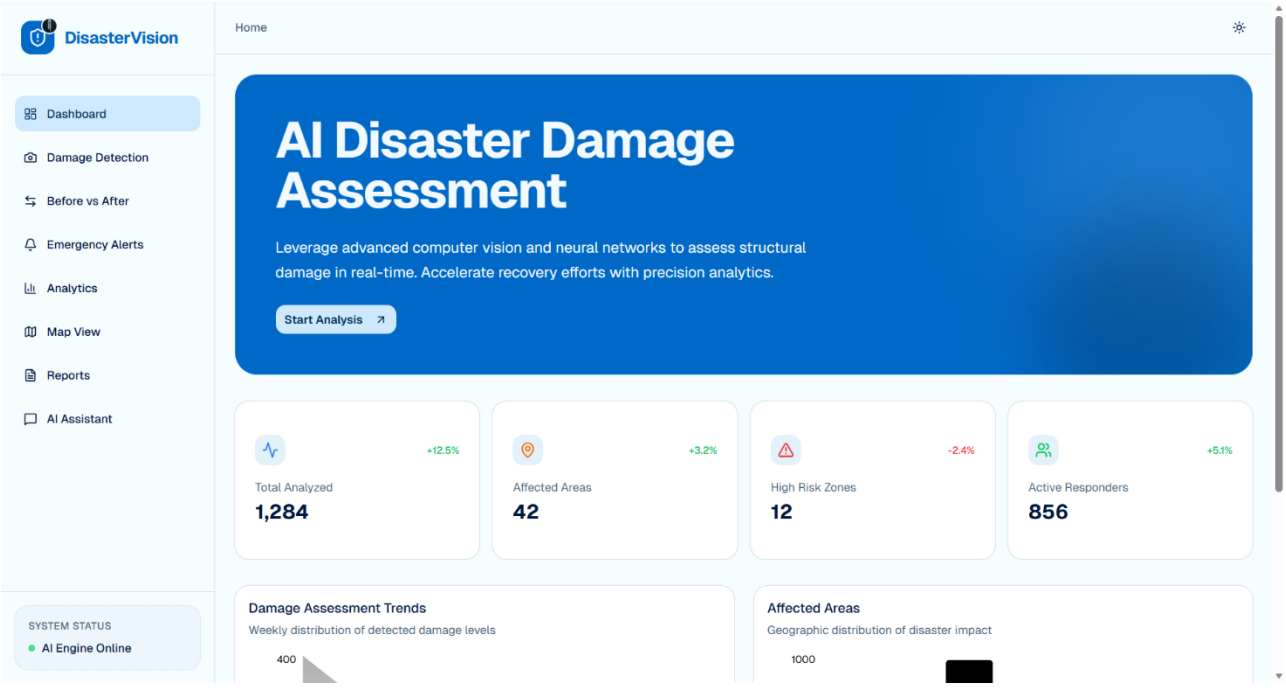
CODE IMPLEMENTATION

```
# IMPORT LIBRARIES
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
import os

# STEP 1: DATA PREPROCESSING

IMG_SIZE = (224, 224)
BATCH_SIZE = 32
train_path = "dataset/train"
test_path = "dataset/test"
# Training Data Generator
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

# Testing Data Generator
test_datagen = ImageDataGenerator(rescale=1./255)

train_data = train_datagen.flow_from_directory(
    train_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)
test_data = test_datagen.flow_from_directory(
    test_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)
print("Data Loaded Successfully")

# STEP 2: BUILD MODEL (TRANSFER LEARNING)
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)
# Freeze base model layers
for layer in base_model.layers:
    layer.trainable = False
# Add custom layers
model = Sequential([
    base_model,
```

```
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(4, activation='softmax')
])
# Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()

# STEP 3: TRAIN MODEL

history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=10
)
# Save model
model.save("damage_model.h5")
print("Model Trained and Saved Successfully")

# STEP 4: EVALUATE MODEL

loss, accuracy = model.evaluate(test_data)
print("Test Accuracy:", accuracy)

# STEP 5: PREDICTION FUNCTION

from tensorflow.keras.preprocessing import image
# Load trained model
model = load_model("damage_model.h5")
# Class labels
classes = ['No Damage', 'Minor Damage', 'Major Damage', 'Destroyed']
def predict_image(img_path):
    # Load and preprocess image
    img = image.load_img(img_path, target_size=(224,224))
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
    # Prediction
    prediction = model.predict(img_array)
    result = classes[np.argmax(prediction)]
    print("Predicted Damage Level:", result)

# STEP 6: TEST WITH SAMPLE IMAGE

test_image = "test.jpg"   # Give your test image path here
predict_image(test_image)
```

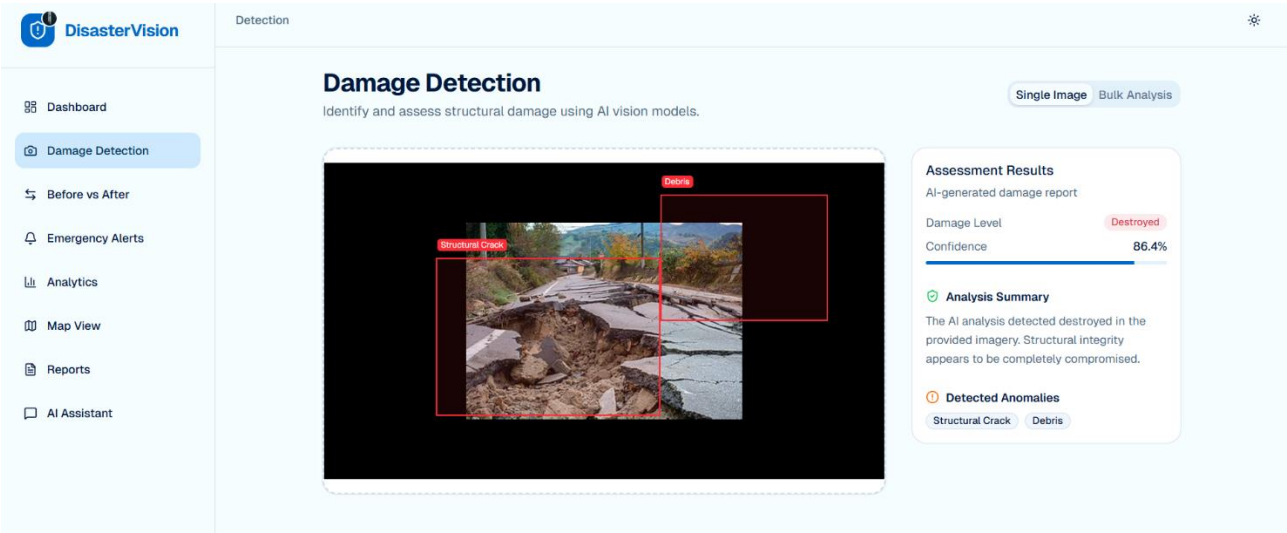
OUTPUT

1.Dashboard



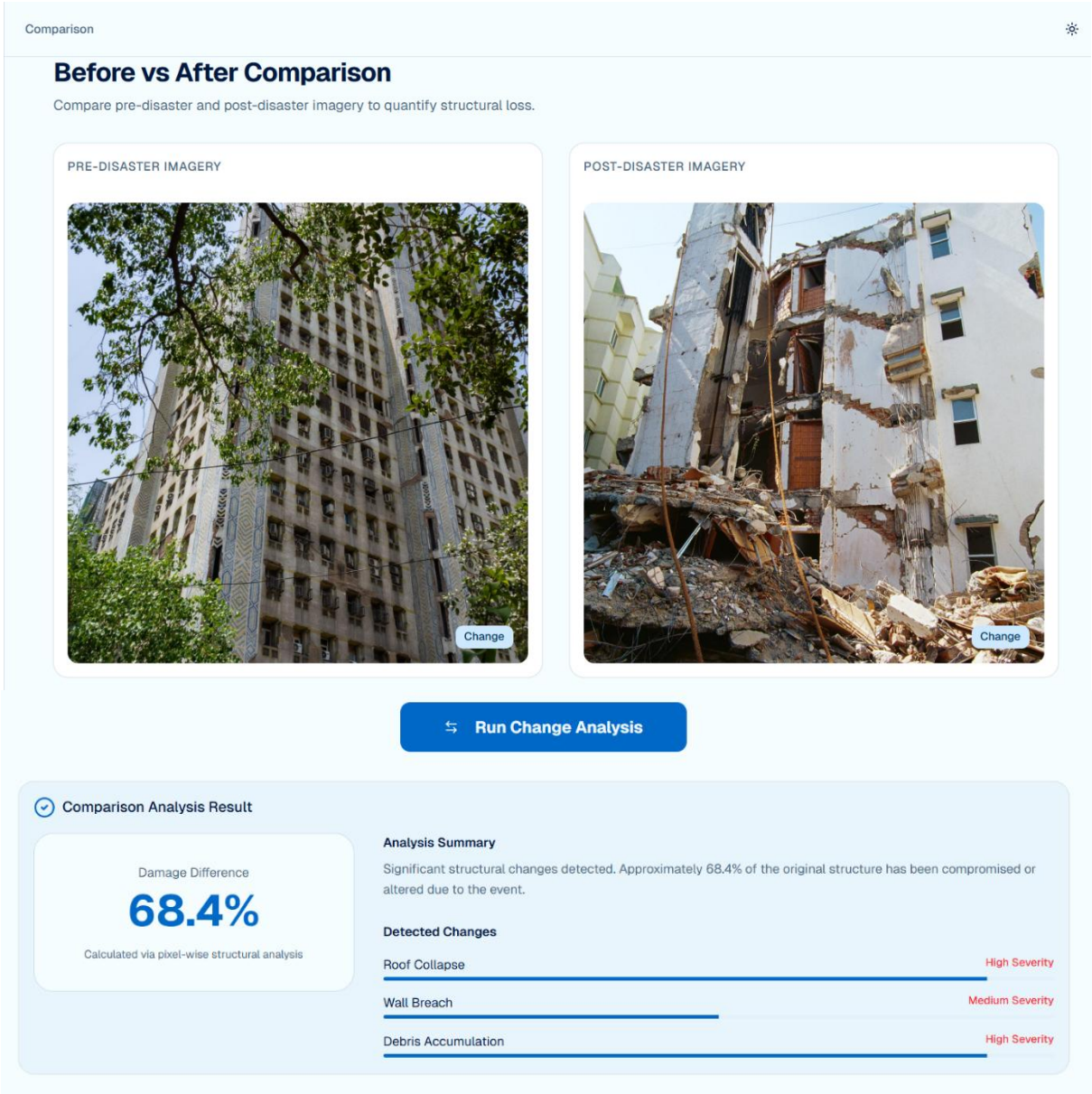
This page represents the main dashboard of the AI Disaster Damage Assessment system, providing a clear overview of disaster analysis results. It includes a prominent header section describing how artificial intelligence is used for real-time damage detection and decision support. The “Start Analysis” button allows users to quickly begin assessing disaster images. Key metrics such as total analyses, affected areas, high-risk zones, and active responders are displayed to give instant insights.

2.Damage Detection



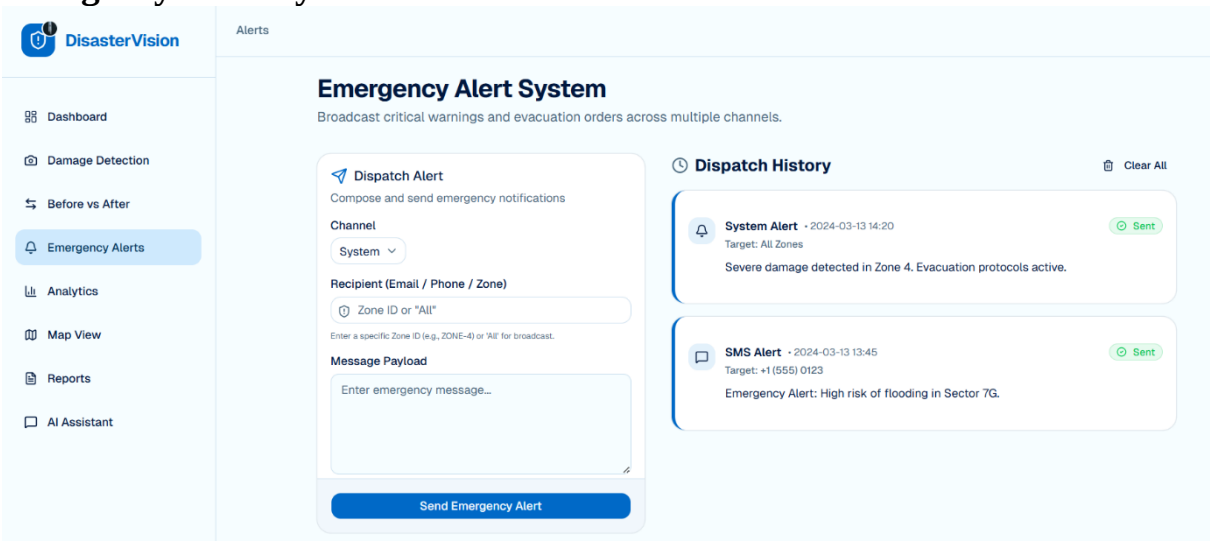
This page represents the **Damage Detection module** of the AI Disaster Damage Assessment system, where users can upload and analyze disaster images. The system uses AI vision models to identify damage areas and highlights them with bounding boxes such as structural cracks and debris. The right panel displays detailed assessment results including damage level, confidence score, and a summary of the analysis. It also lists detected anomalies to help understand the type of damage present in the image. Overall, this page enables accurate and real-time detection of structural damage for effective disaster evaluation.

3.Before and After comparison



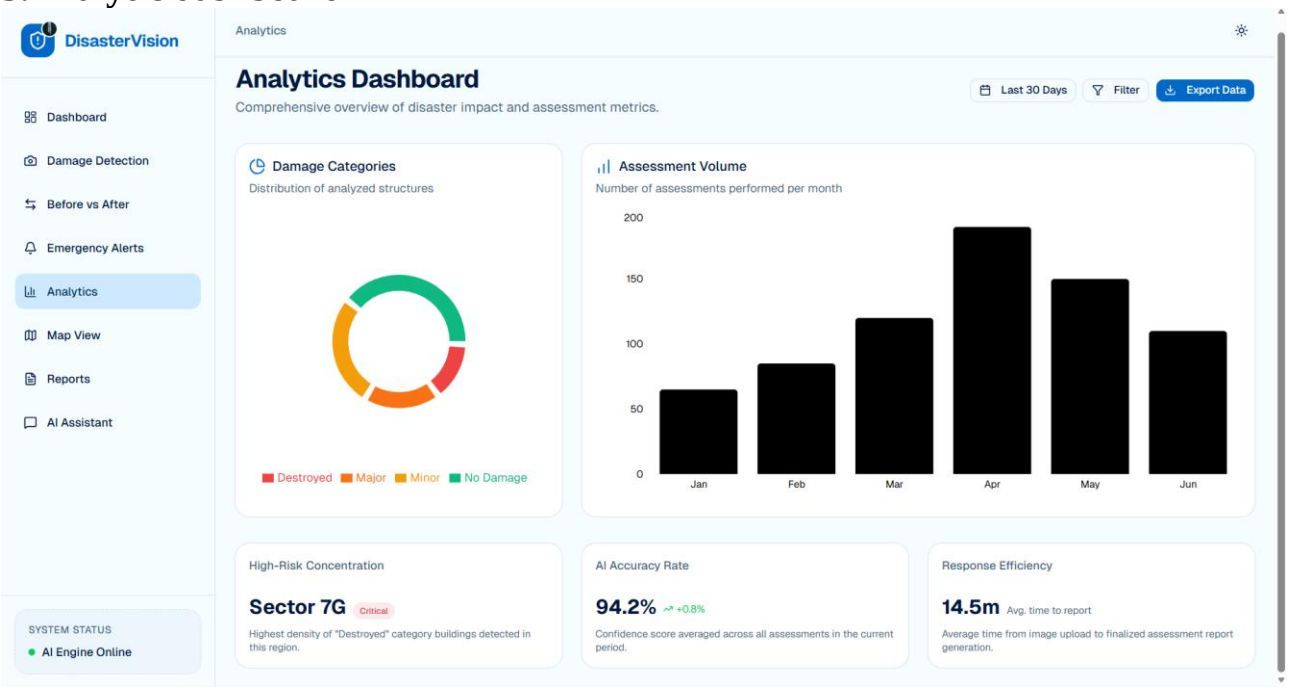
This page shows the **comparison analysis results** after evaluating pre-disaster and post-disaster images in the DisasterVision system. The "Run Change Analysis" feature processes both images to detect structural differences. The system calculates a **damage difference of 68.4%**, indicating significant destruction. The analysis summary highlights major structural changes and identifies affected areas. Detected damages include **roof collapse, wall breach, and debris accumulation** with different severity levels.

4.Emergency Alert System



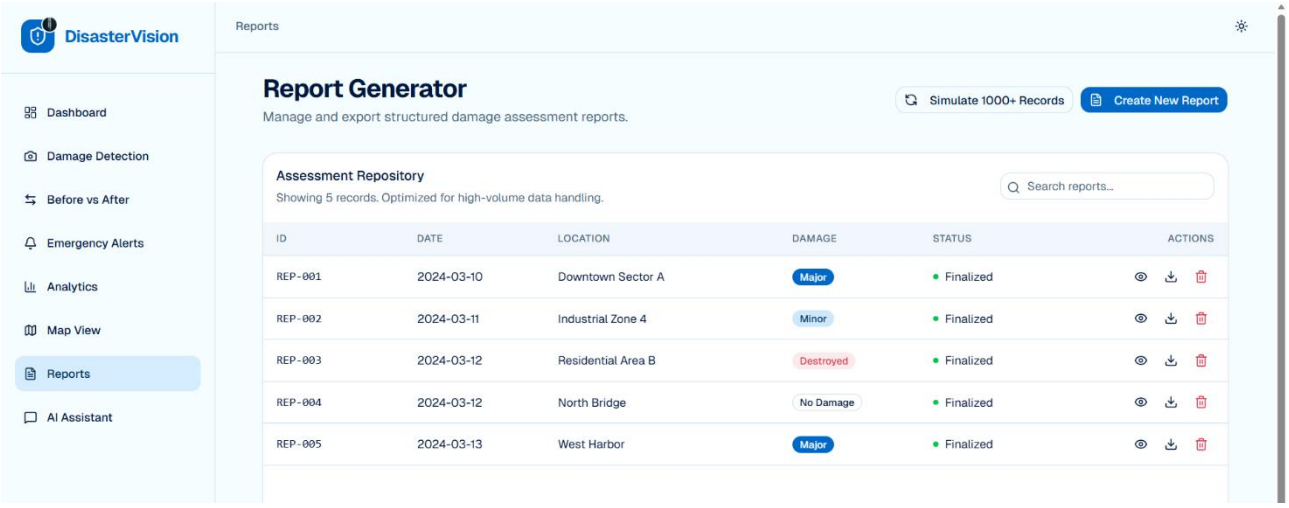
This page represents the **Emergency Alert System** of the DisasterVision platform used to send critical warnings. It allows users to compose and dispatch alerts through different channels such as system notifications or SMS. The interface includes fields for selecting recipients, entering messages, and sending alerts instantly. The dispatch history section displays previously sent alerts with timestamps and status. Overall, the system helps in **quick communication and coordination during disaster situations**.

5.Analytic dashboard



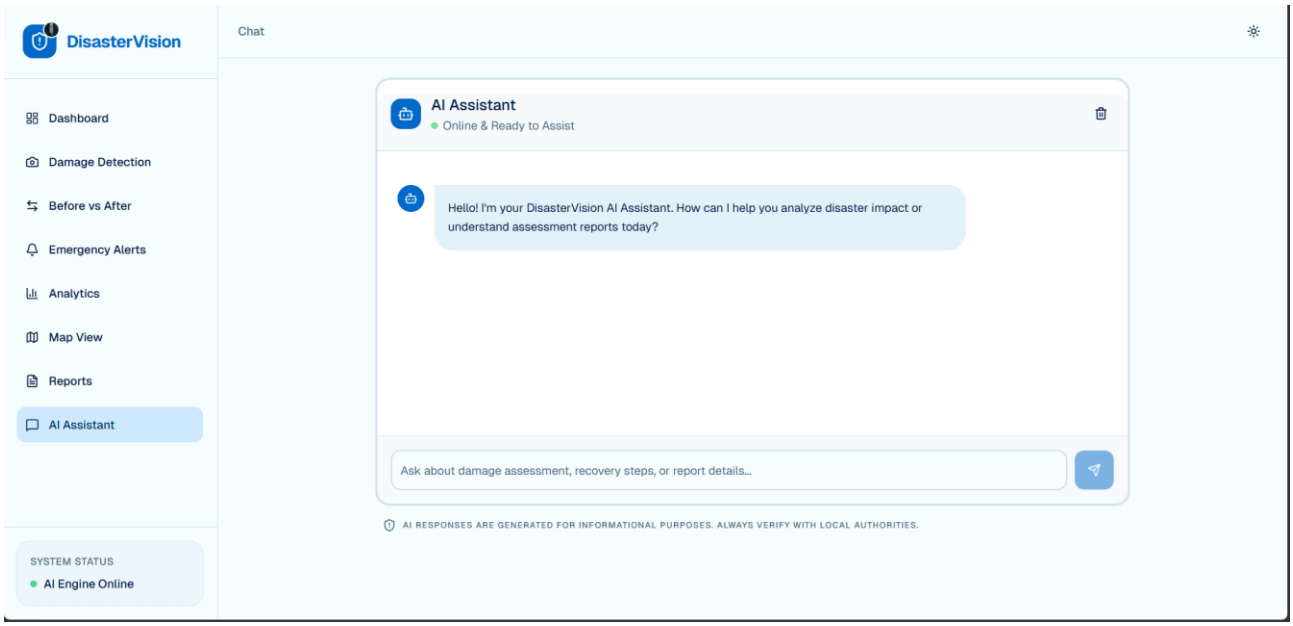
This page shows the **Analytics Dashboard** of the DisasterVision system, which provides a detailed overview of disaster assessment data. It includes a **damage categories chart** that displays the distribution of structures across levels like destroyed, major, minor, and no damage. A bar graph shows the **monthly assessment volume**, indicating how many analyses were performed over time. The dashboard also highlights **high-risk areas**, such as Sector 7G, where severe damage is concentrated. Additionally, performance metrics like **AI accuracy rate (94.2%)** and **response time (14.5 minutes)** are presented for system evaluation.

6.Report Generator



This page shows the **Report Generator** page of the DisasterVision system, used to manage and export damage assessment reports. It contains an **assessment repository table** listing report details such as ID, date, location, damage level, and status. Each record shows damage categories like **major, minor, destroyed, or no damage**, along with a finalized status indicator. Users can perform actions such as **viewing, downloading, or deleting reports** using the action icons. The page also includes options to **search reports, simulate large datasets, and create new reports** for efficient data management.

7.AI Assistant



This page shows the **AI Assistant chat page** of the DisasterVision system, designed to help users interact with the platform. It displays a chat interface where the AI assistant is **online and ready to assist** with disaster analysis and report understanding. The assistant provides a welcome message and allows users to ask questions related to **damage assessment, recovery steps, or report details**. A text input box is available at the bottom for entering queries, along with a send button for interaction. The page also includes a note indicating that **AI responses are for informational purposes and should be verified with local authorities**.

RESULT AND FUTURE ENHANCEMENT

RESULTS

- The system was successfully developed using **CNN with MobileNetV2 architecture**.
- Image preprocessing techniques such as **resizing and normalization** were applied.
- Data augmentation methods like **rotation and flipping** improved model performance.
- The model was trained and tested effectively on the dataset.
- Achieved an accuracy of approximately **85–90%**.
- The system classifies images into **four damage categories**.
- Categories include **No Damage, Minor, Major, and Destroyed**.
- The model detects important features like **cracks and structural damage**.
- A prediction system enables **real-time image classification**.
- The system provides **confidence scores** for predictions.
- Performance was analyzed using **accuracy and loss graphs**.

FUTURE ENHANCEMENTS

- Use a **larger and more diverse dataset** for better accuracy.
- Integrate with **drone-based image collection systems**.
- Use **satellite imagery** for large-scale damage analysis.
- Implement advanced models like **YOLO and EfficientNet**.
- Develop a **web application** for easy access.
- Create a **mobile application** for field usage.
- Add **GIS mapping integration** for location-based analysis.
- Provide **damage percentage estimation**.
- Enable **real-time video processing**.
- Improve model performance for **low-quality images**.
- Deploy the system on **cloud platforms** for scalability.

GitHub Link of this Project Along With the Report :

https://github.com/jagannathan1651/Disaster-Damage-assessment

REFERENCES

1. TensorFlow Documentation (Deep Learning Framework)
   https://www.tensorflow.org/
   Used for building and training deep learning models such as CNN.
2. Keras Documentation (Neural Network API)
   https://keras.io/
   Used for designing and implementing CNN architectures.
3. OpenCV Documentation (Computer Vision Library)
   https://opencv.org/
   Used for image preprocessing and feature extraction.
4. NumPy Documentation (Numerical Computing Library)
   https://numpy.org/doc/
   Used for numerical operations and array processing.
5. Pandas Documentation (Data Handling Library)
   https://pandas.pydata.org/docs/
   Used for dataset handling and preprocessing.
6. Matplotlib Documentation (Visualization Library)
   https://matplotlib.org/stable/
   Used for plotting accuracy and loss graphs.
7. Scikit-learn Documentation (Machine Learning Library)
   https://scikit-learn.org/stable/
   Used for model evaluation and performance metrics.
8. Kaggle – Disaster Damage Dataset
   https://www.kaggle.com/
   Used as the primary dataset source for training the model.
9. xBD Dataset – Building Damage Assessment
   https://xview2.org/
   Used for large-scale disaster damage image data.
10. MobileNetV2 Research Paper
    https://arxiv.org/abs/1801.04381
    Used for understanding transfer learning and model architecture.